

Sztuczna inteligencja

Implementacja detekcji obiektów z wykorzystaniem algorytmu YOLOv5



POLITECHNIKA
LUBELSKA
WYDZIAŁ ELEKTROTECHNIKI
I INFORMATYKI

Autor: Kondraciuk Jakub

Spis treści

1. Założenia projektu	3
2. Wstęp teoretyczny	3
2.1 Detekcja obiektów	3
2.2 Rodzina algorytmów YOLO	3
2.3 YOLOv5 architektura i zasada działania	3
3. Przygotowanie bazy obrazów i etykiet	4
3.1 Pobieranie obrazów	4
3.2 Etykietowanie obrazów	5
4. Uczenie modelu	8
4.1 Przygotowanie środowiska	8
4.2 Trenowanie modelu	8
4.3 Ocena wyników treningu	9
5. Testowanie modelu	11
5.1 Parametry testowania	11
5.2 Wyniki – przykładowe obrazy	11
6. Podsumowanie	18

1. Założenia projektu

Projekt obejmuje stworzenie modelu systemu detekcji obiektów w obrazach z wykorzystaniem algorytmu YOLOv5. W ramach projektu przygotowano zestaw danych składający się ze 200 obrazów, po 50 dla każdej z czterech wybranych klas: pistolet, RPG, karabin szturmowy oraz karabin snajperski. Celem było wytrenowanie modelu YOLOv5 na tym zbiorze oraz przeprowadzenie testów mających na celu ocenę skuteczności wykrywania poszczególnych klas broni na nowych obrazach. Projekt obejmował etapy przygotowania i anotacji danych, trenowania modelu, jego ewaluacji oraz analizy wyników.

2. Wstęp teoretyczny

2.1 Detekcja obiektów

Detekcja obrazów polega na lokalizacji i klasyfikacji występujących w obrazie obiektów. W odróżnieniu od klasyfikacji obrazów, która przypisuje pojedynczą etykietę całemu obrazowi, detekcja pozwala wskazać nie tylko co znajduje się na obrazie, ale również gdzie się znajduje.

2.2 Rodzina algorytmów YOLO

Pierwsze algorytmy detekcji obrazów opierały się na metodach wykorzystujących inżynierię cech, takich jak HOG, SIFT czy Haar Cascades. Jednak metody te były mało elastyczne i nie radziły sobie z różnorodnymi danymi. Pojawienie się głębokich sieci neuronowych, zrewolucjonizowało podejście do tego zagadnienia. Pierwsze nowoczesne algorytmy detekcji oferowały wysoką dokładność, ale były relatywnie wolne w działaniu. W odpowiedzi na potrzebę szybszych systemów detekcji powstała rodzina algorytmów YOLO. Są to algorytmy zaprojektowane z myślą o szybkości działania bez znaczącego pogorszenia dokładności. Nazwa "You Only Look Once" odzwierciedla główną ideę algorytmu: obraz analizowany jest w całości w pojedynczym przebiegu sieci neuronowej.

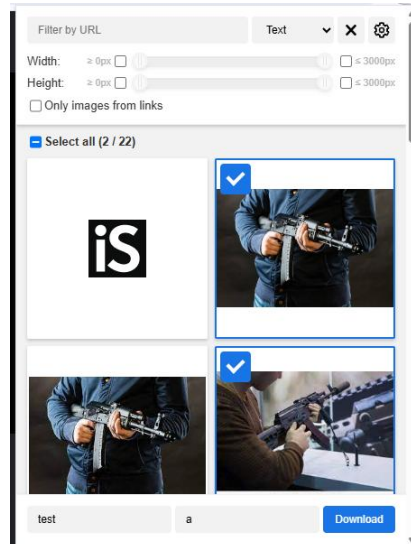
2.3 YOLOv5 architektura i zasada działania

- Model YOLOv5 składa się z trzech głównych części:
 1. **Backbone** – odpowiedzialny za ekstrakcję cech z obrazu
 2. **Neck** – pośredni komponent agregujący informacje z różnych poziomów abstrakcji. Umożliwia on lepsze wykrywanie obiektów o różnych rozmiarach
 3. **Head** – końcowa część odpowiedzialna za predykcję bounding boxów, klas oraz confidence score
- Proces działania YOLOv5:
 1. **Preprocessing** - obraz wejściowy jest przeskalowywany i normalizowany
 2. **Ekstrakcja cech** - przez kolejne warstwy konwolucyjne wydobywane są coraz bardziej abstrakcyjne cechy z obrazu
 3. **Agregacja cech** - informacje z różnych poziomów sieci są agregowane, aby umożliwić wykrywanie zarówno małych, jak i dużych obiektów
 4. **Predykcja** - dla każdej komórki siatki model przewiduje zestaw bounding boxów wraz z przypisanymi klasami i confidence score
 5. **Postprocessing** - usuwanie nakładających się predykcji, pozostawiając te z najwyższym prawdopodobieństwem

3. Przygotowanie bazy obrazów i etykiet

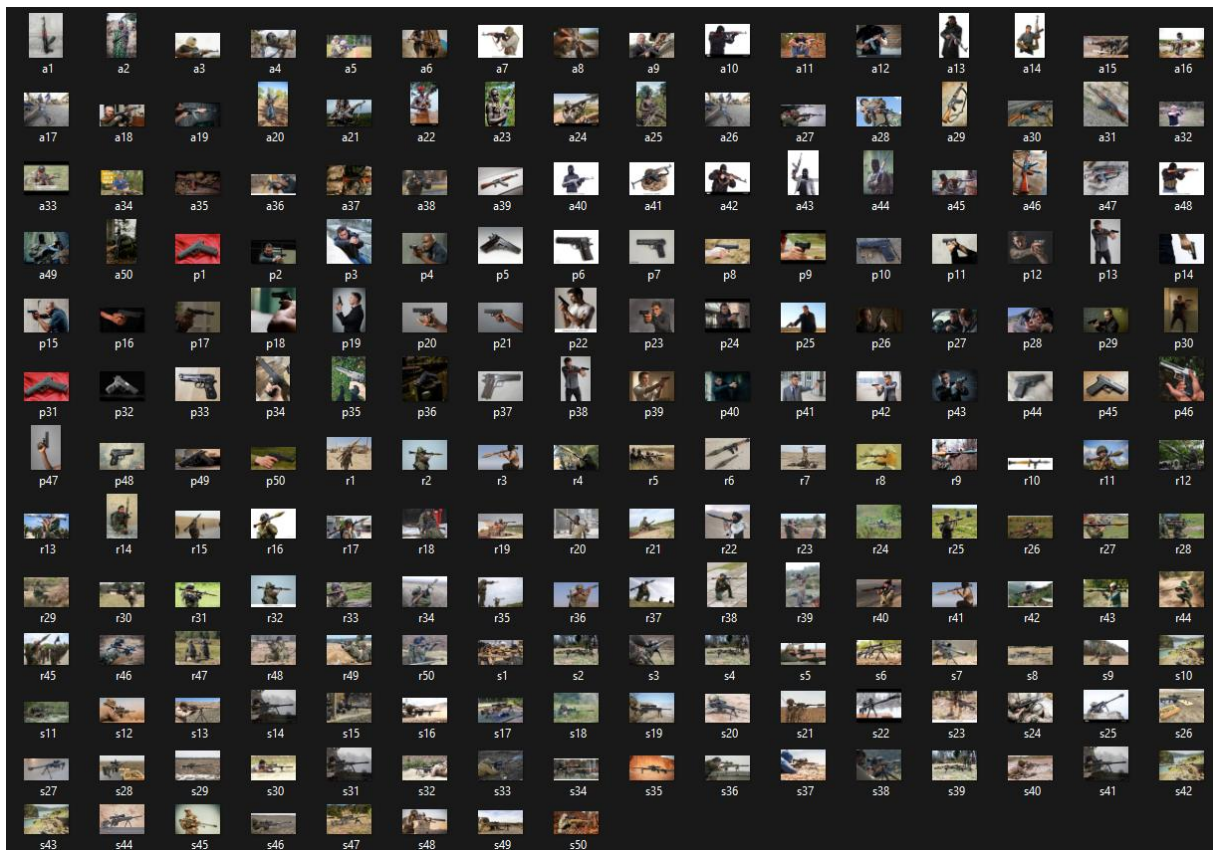
3.1 Pobieranie obrazów

Do pozyskania obrazów wykorzystano rozszerzenie przeglądarki Google Chrome – Image Downloader. Jest to narzędzie umożliwiające pobieranie wielu obrazów jednocześnie z wyników wyszukiwania w Google Images.



Rys. 1 Image Downloader

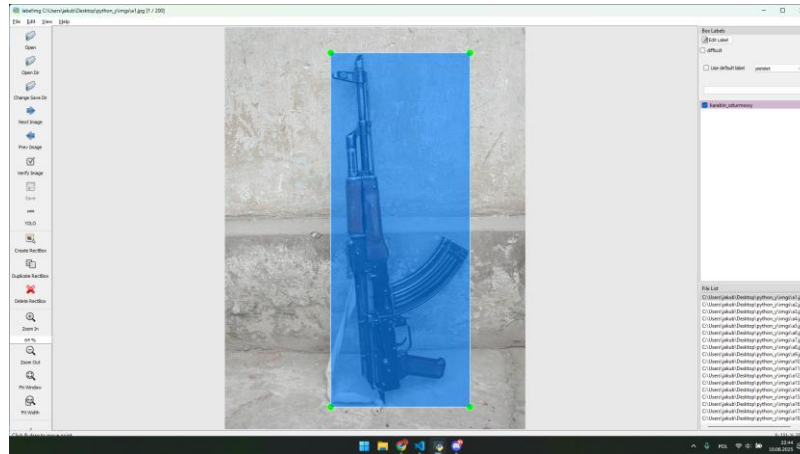
Łącznie pobrano 200 zdjęć, po 50 sztuk dla każdej z 4 klas.



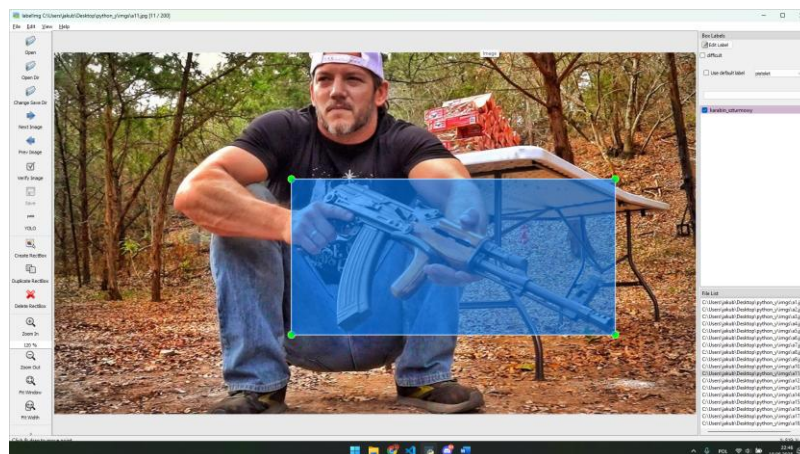
Rys. 2 Baza obrazów

3.2 Etykietowanie obrazów

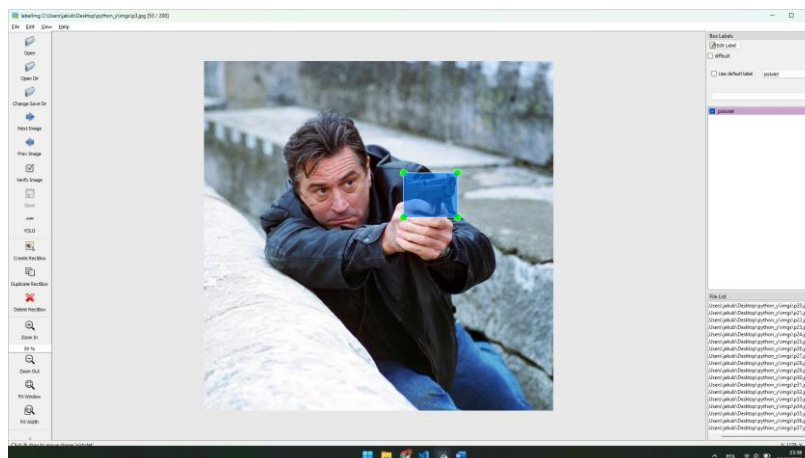
Po zebraniu obrazów konieczne było ich ręczne etykietowanie, czyli oznaczenie, gdzie na obrazie znajdują się obiekty należące do poszczególnych klas. W tym celu wykorzystano popularne narzędzie LabelImg.



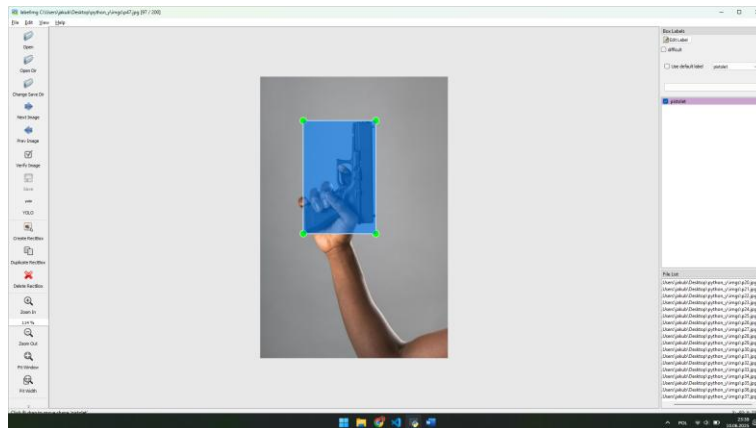
Rys. 3 Etykietowanie - przykład 1 (karabin szturmowy)



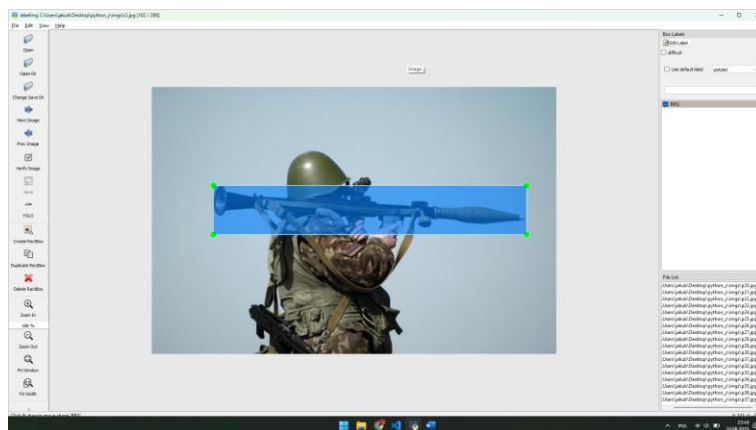
Rys. 4 Etykietowanie - przykład 2 (karabin szturmowy)



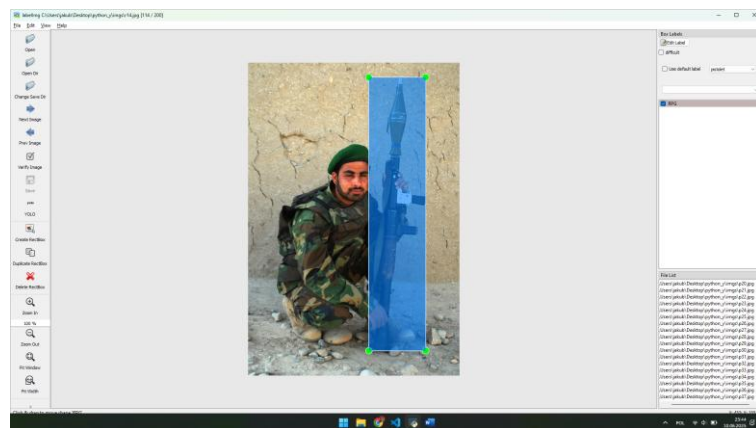
Rys. 5 Etykietowanie - przykład 3 (pistolet)



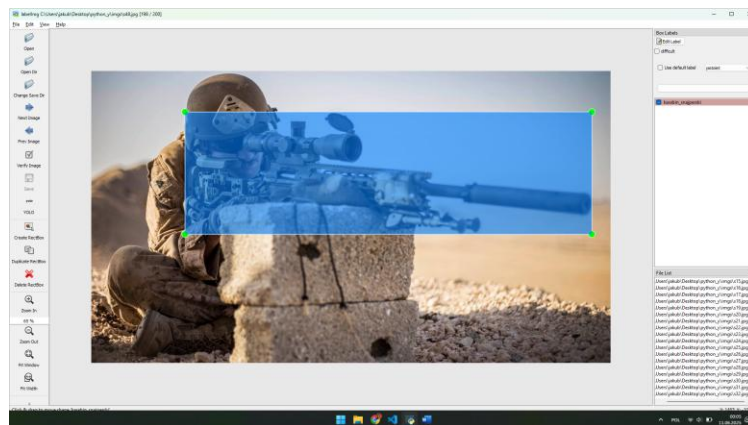
Rys. 6 Etykietowanie - przykład 4 (pistolet)



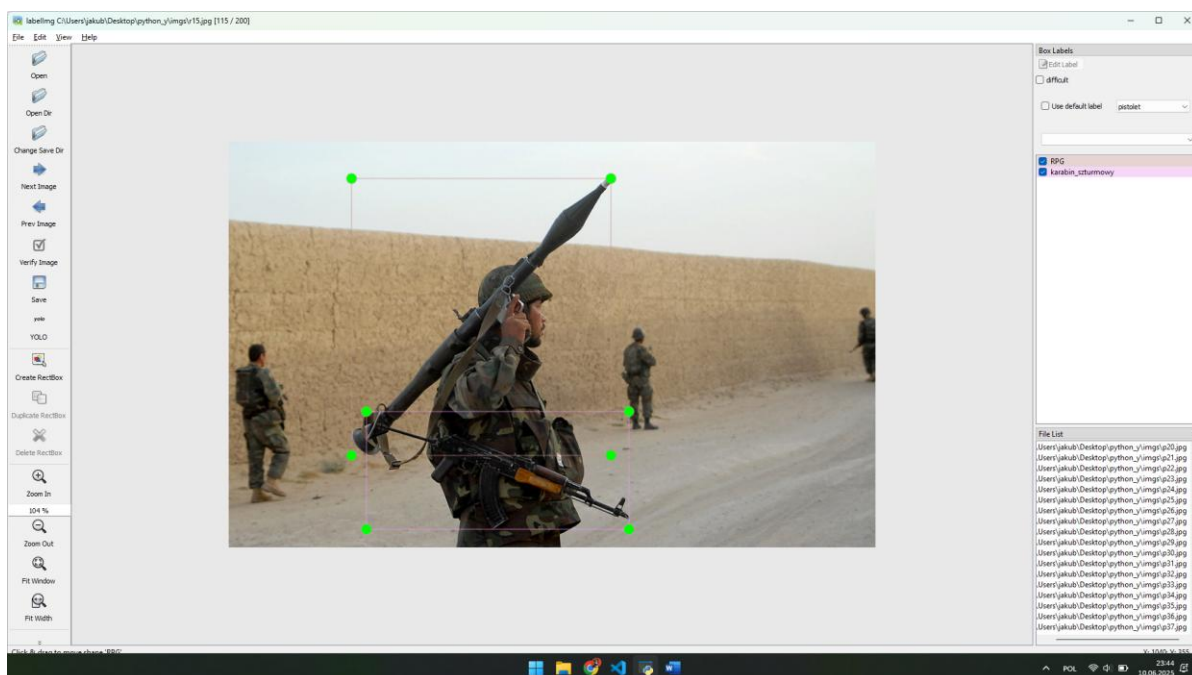
Rys. 7 Etykietowanie - przykład 5 (RPG)



Rys. 8 Etykietowanie - przykład 6 (RPG)



Rys. 9 Etykietowanie - przykład 7 (karabin snajperski)



Rys. 10 Etykietowanie - przykład 8 (dwa obiekty na jednym obrazie)

Pojedyncza etykieta:

Plik pojedynczej etykiety ma rozszerzenie „.txt” i ma postać:

<class_id> <x_center> <y_center> <width> <height>

- class_id - mówi do jakiej klasy należy obiekt
- x_center - pozycja środka ramki w osi X
- y_center - pozycja środka ramki w osi Y
- width - szerokość ramki jako ułamek szerokości obrazu
- height - wysokość ramki jako ułamek wysokości obrazu

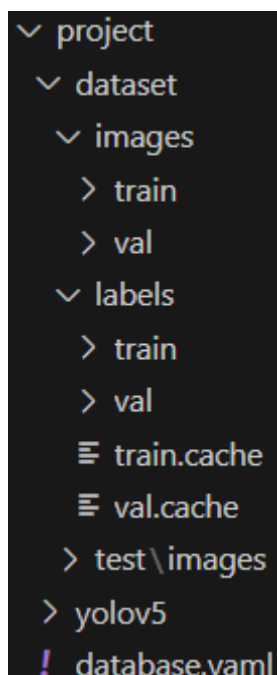
2 0.584075 0.504000 0.463523 0.881333

Rys. 11 Przykładowa etykieta

4. Uczenie modelu

4.1 Przygotowanie środowiska

Utworzona została odpowiednia struktura katalogów, gdzie w głównym folderze znajdują się katalog „dataset”, który zawiera obrazy i etykiety, podzielone w stosunku 80% - trening, 20% - walidacja. Katalog yolov5, w którym znajduje się model oraz plik „database.yaml”, który przechowuje konfigurację modelu.



Rys. 12 Struktura katalogów

```
train: ../../dataset/images/train
val: ../../dataset/images/val

nc: 4
names: ['pistolet', 'RPG', 'karabin_szturmowy', 'karabin_snajperski']
```

Rys. 13 Zawartość pliku "database.yaml"

4.2 Trenowanie modelu

Parametr	Znaczenie
--img 640	Obrazy wejściowe są skalowane do rozdzielczości 640×640 pikseli
--batch 16	16 obrazów jest przetwarzanych równocześnie w jednej iteracji
--epochs 150	Model przejdzie przez cały zbiór treningowy 150 razy
--data ../../database.yaml	Ścieżka do pliku .yaml zawierającego konfigurację danych
--weights yolov5s.pt	Start treningu z wagami pretrenowanego modelu YOLOv5s
--cache	Obrazy i etykiety są buforowane w pamięci RAM dla przyspieszenia treningu
--patience 20	Jeśli mAP@0.5 nie poprawia się przez 20 epok, trening kończy się wcześniej
--name moj_model	Wszystkie wyniki będą zapisane w runs/train/moj_model
--project runs/train	Ścieżka do folderu, w którym zostanie utworzony folder z wynikami treningu
--exist-ok	Jeśli folder runs/train/moj_model już istnieje, będzie zaktualizowany

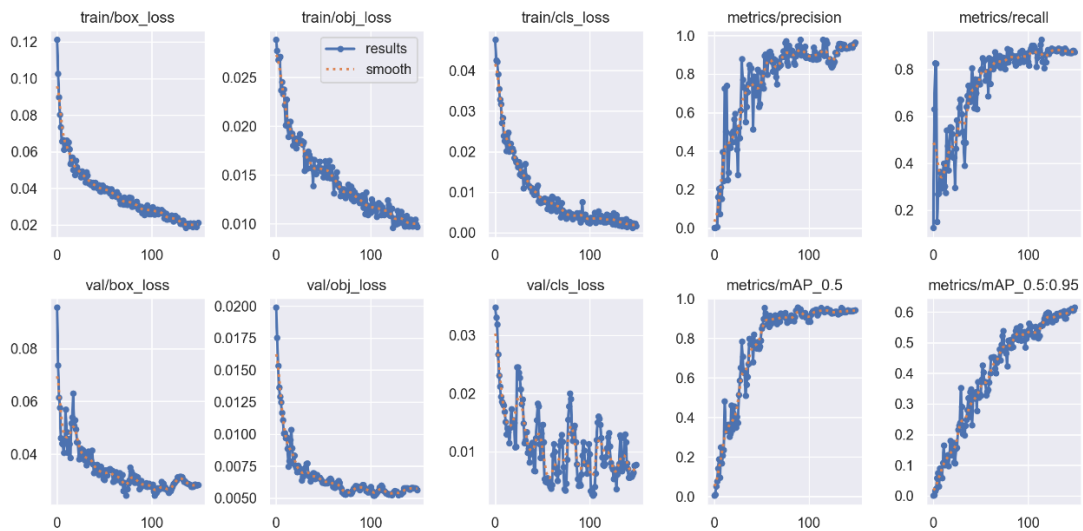
Tab. 1 Parametry trenowania modelu

4.3 Ocena wyników treningu

Wszystkie funkcje straty zarówno dla treningu jak i walidacji wykazują prawidłową tendencję spadkową i stabilizują się, co wskazuje na skuteczne uczenie się modelu. Model osiąga bardzo dobre wyniki:

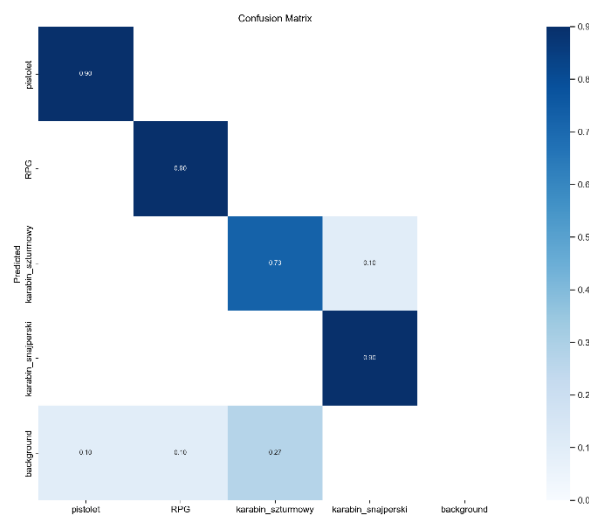
- Precision: ~ 0.95
- Recall: ~ 0.9
- mAP@0.5: ~ 0.93
- mAP@0.5:0.95: ~ 0.6

Model wydaje się być delikatnie przeuczony: Validation loss wykazuje fluktuacje i wzrost w późniejszych epokach, a różnica między train a validation loss zwiększa się z czasem



Rys. 14 Zbiór wykresów z pliku "results.png"

Bardzo wysoka dokładność klasyfikacji klas: Pistolet: 90%, RPG: 90%, Karabin snajperski: 90%. Model praktycznie nie myli między sobą klas broni, co wskazuje na bardzo dobre wyuczenie się charakterystycznych cech każdej kategorii. Trzeba zaznaczyć, że model przeprowadzał walidację na 10 obrazach dla każdej klasy, co może nie być do końca miarodajne.



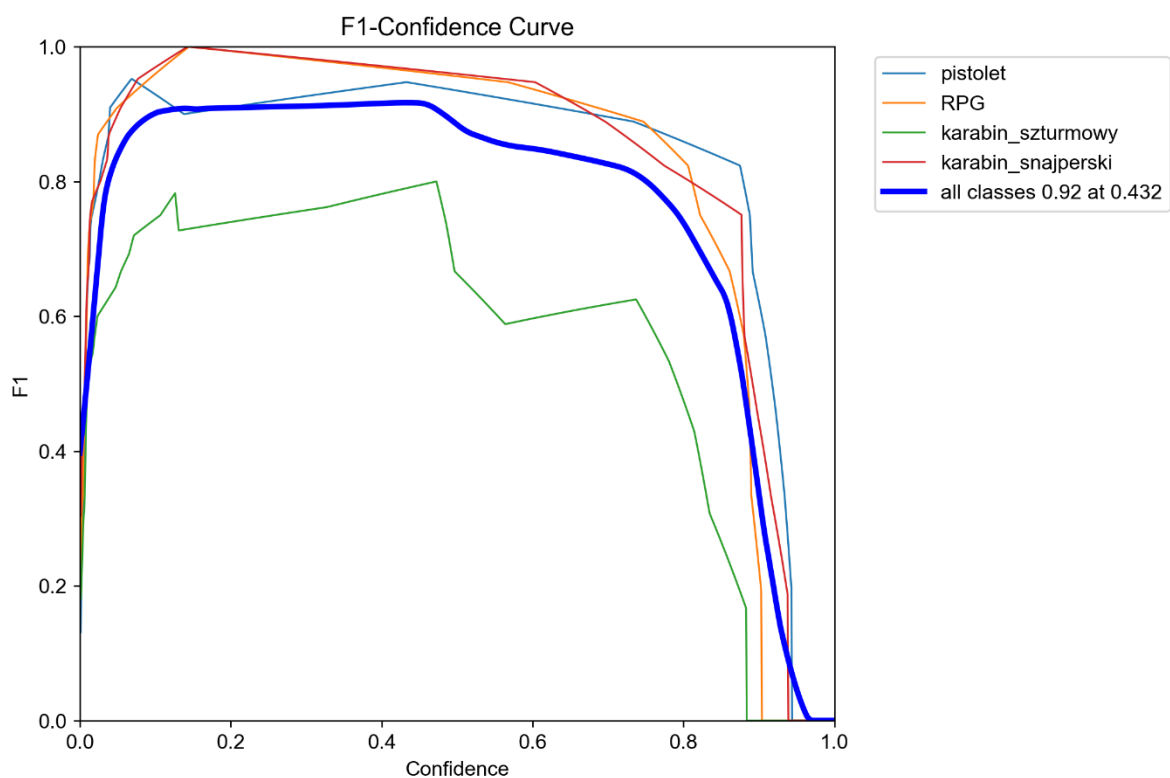
Rys. 15 Macierz pomyłek

Najlepsze klasy:

- **Karabin snajperski** - niemal idealna krzywa, $F1 \approx 1.0$ w zakresie confidence (0.1-0.8)
- **RPG** - bardzo dobra wydajność, $F1 \approx 0.98$ w zakresie 0.1-0.6
- **Pistolet** - solidna wydajność, $F1 \approx 0.95$ w zakresie 0.2-0.5

Problematiczna klasa:

- **Karabin szturmowy** - wyraźnie gorsza od reszty:
 - Maksymalny $F1 \approx 0.8$
 - Bardzo niestabilna krzywa z wieloma spadkami
 - Słaba wydajność przy wyższych wartościach confidence



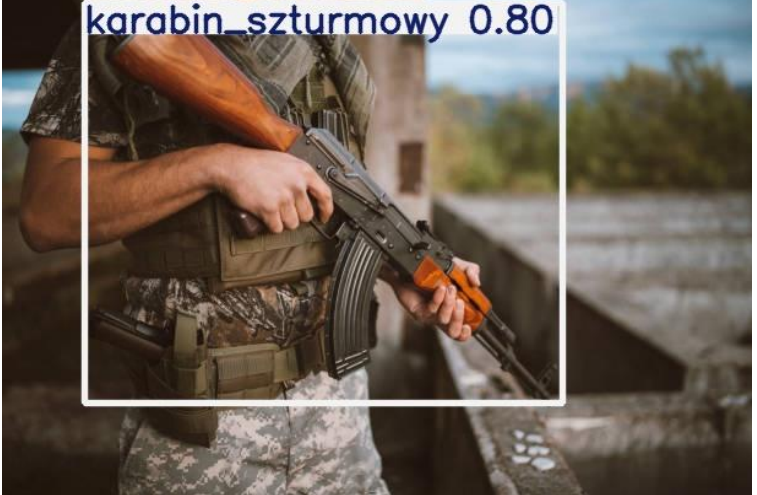
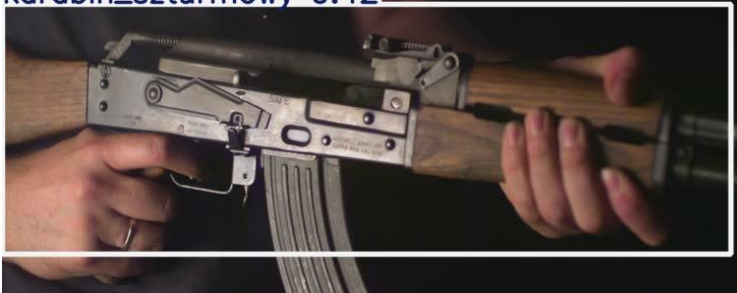
Rys. 16 Krzywa F1

5. Testowanie modelu

5.1 Parametry testowania

Pobrano 200 nowych zdjęć, w losowych proporcjach klas i przekazano je do modelu. Parametr conf, czyli confidence, został ustawiony na 0.1

5.2 Wyniki – przykładowe obrazy

Obraz	Opis
	Poprawna detekcja.
	Poprawna detekcja.
	Poprawna detekcja.

 karabin_szturmowy 0.29	Poprawna detekcja.
 RPG 0.53	Niepoprawna detekcja – model pomylił 2 karabiny szturmowe z RPG.
 RPG 0.15 in_szturmowy 0.77	Częściowo poprawna detekcja – model poprawnie wykrył karabin szturmowy, natomiast pomylił element tła z RPG.
 in_szturmowy 0.74 in_szturmowy 0.64	Poprawna detekcja.

	Poprawna detekcja.
	Poprawna detekcja.
	Poprawna detekcja.
	Częściowo poprawna detekcja – model wykrył jeden karabin snajperski, natomiast na obrazie znajduje się jeszcze kilka mniej widocznych karabinów snajperskich.

	Poprawna detekcja.
	Częściowo poprawna detekcja – model poprawnie wykrył jeden pistolet, natomiast nie wykrył drugiego i pomylił cień z pistoletem.
	Poprawna detekcja.
	Poprawna detekcja.

 shutterstock.com · 1417894409	Poprawna detekcja.
	Częściowo poprawna detekcja – model poprawnie wykrył dwa pistolety, natomiast pomylił cień z pistoletem.
	Brak detekcji.
	Poprawna detekcja.
	Poprawna detekcja.

		Częściowo poprawna detekcja – model poprawnie wykrył RPG, natomiast pomylił gitarę z karabinem szturmowym.
		Niepoprawna detekcja – model pomylił RPG z karabinem snajperskim.
		Poprawna detekcja.
		Poprawna detekcja.
		Poprawna detekcja.

		Poprawna detekcja.
		Poprawna detekcja.
		Poprawna detekcja.
		Niepoprawna detekcja – model pomylił RPG z karabinem szturmowym.
		Częściowo poprawna detekcja – model wykrył dwa z pięciu RPG.

6. Podsumowanie

Celem projektu było stworzenie systemu rozpoznającego cztery klasy broni: pistolet, RPG, karabin szturmowy oraz karabin snajperski, przy wykorzystaniu ograniczonego zestawu danych (200 obrazów treningowych).

YOLOv5, należący do rodziny jednych z najefektywniejszych algorytmów detekcji obiektów, okazał się trafnym wyborem w kontekście szybkiej i relatywnie precyzyjnej detekcji. Jego architektura, podzielona na backbone, neck i head, umożliwiła skuteczne wyodrębnianie cech i predykcję obiektów nawet w przypadku złożonych obrazów. Analiza działania modelu pokazała, że YOLOv5 potrafi trafnie rozróżniać klasy, nawet gdy różnice wizualne między obiektami są subtelne, jak w przypadku broni palnej.

W fazie testowania wykorzystano 200 nowych zdjęć, co pozwoliło zweryfikować, jak model radzi sobie w bardziej realistycznym scenariuszu. Wyniki testów były zadowalające: W wielu przypadkach model prawidłowo wykrywał obecność i klasę broni. Występowały błędy, zwłaszcza: pomylenie RPG z karabinem szturmowym i odwrotnie, interpretacja tła lub cieni jako obiektów broni, częściowe wykrywanie (np. tylko jednej z kilku broni na obrazie). Mimo to, model wykazywał wysoką odporność na zakłócenia obrazu oraz różnorodność pozycji, rozmiarów i kątów przedstawionych obiektów.

Projekt ukazuje praktyczne zastosowanie zaawansowanych technik sztucznej inteligencji w zadaniu detekcji obiektów. Zrealizowane etapy – od pobierania danych, poprzez ich etykietowanie, trening, ewaluację i testowanie – tworzą kompletny pipeline uczenia maszynowego. Projekt może stanowić solidną bazę do dalszego rozwoju, np. poprzez: rozszerzenie liczby klas, zwiększenie zbioru danych, wdrożenie modelu w środowisku produkcyjnym (np. monitoring wizyjny), badanie wpływu różnych konfiguracji YOLOv5 (np. YOLOv5m, YOLOv5x) na jakość predykcji.